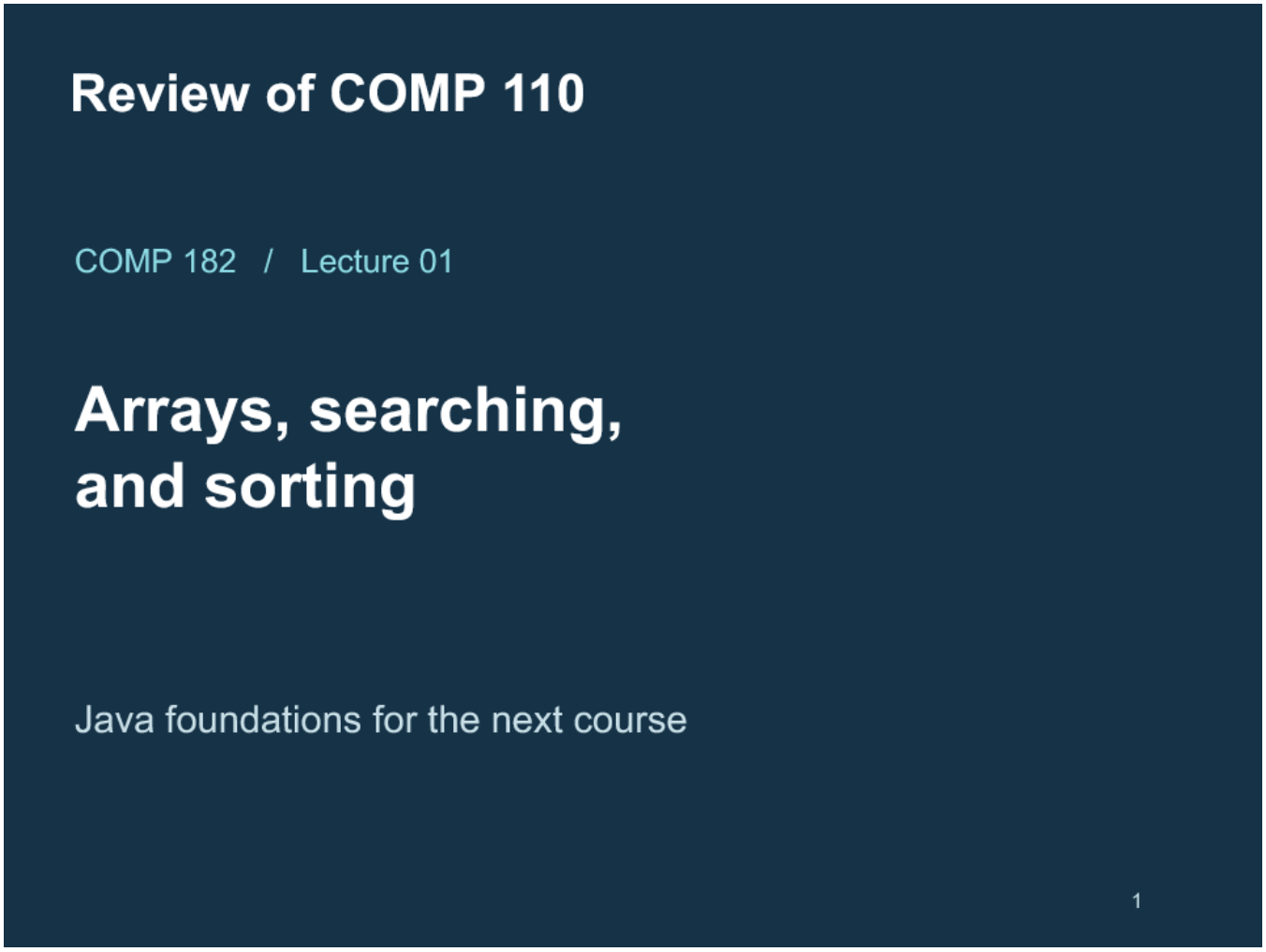


Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Opening prompt: What is the difference between an array variable and the array itself? This review develops that distinction through examples.

Review objectives

Create and process one-dimensional arrays.

Explain references, copies, and method arguments.

Trace linear search, binary search, and selection sort.

Work with rectangular and ragged two-dimensional arrays.

2

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Declaring and creating an array

```
int[] scores;              // declare a reference
scores = new int[4];        // create the array

int[] values = new int[4];
```

Index	0	1	2	3
Initial value	0	0	0	0

The length stays fixed after creation.

3

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Reference: Oracle Java Tutorials, Arrays.
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html
A local reference variable must receive a value before use. Creating an array allocates a fixed number of elements. An int[] stores int values.

Indexes, length, and default values

```
int[] a = new int[4];
a[0] = 12;
a[a.length - 1] = 27;
```

Element type	Default in a new array
int / double	0 / 0.0
boolean	false
char	\u0000
Reference type	null

a.length is 4. a[4] throws ArrayIndexOutOfBoundsException.

4

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Valid indexes are 0 through length−1. A zero-length array has no valid element index. A local array variable does not automatically refer to an allocated array. Reference: Oracle Java Tutorials, Arrays. https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

Array initializers

```
double[] values = {1.9, 2.9, 3.4, 3.5};

// Later reassignment needs an explicit new array.
values = new double[] {2.0, 4.0, 6.0};
```

An initializer sets the length from its element count.

5

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The brace-only initializer syntax belongs in a declaration. For later assignment, use a new-array expression. These two arrays have the same values but are separate objects.

Input, random values, and display

```
// Fill from a Scanner named input.
for (int i = 0; i < a.length; i++)
    a[i] = input.nextInt();

// Or fill with random integers from 0 through 99.
for (int i = 0; i < a.length; i++)
    a[i] = rng.nextInt(100);

for (int value : a)
    System.out.println(value);
```

A traversal visits each element once.

6

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Three alternative operations on int[] a. For input, assume Scanner input = new Scanner(System.in). For random values, assume java.util.Random rng = new java.util.Random(). nextInt(100) returns an integer from 0 inclusive to 100 exclusive.

Sum and first maximum index

```
long total = 0;
for (int value : a) total += value;

int best = 0;
for (int i = 1; i < a.length; i++) {
    if (a[i] > a[best]) best = i;
}

System.out.println(total);
System.out.println("max = " + a[best]);
System.out.println("index = " + best);
```

Initialize the maximum from an actual element.

7

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume a is non-null and nonempty. Using > rather than >= preserves the first index when maximum values tie. To find the minimum, reverse the comparison. long total reduces the risk of overflow when summing int elements.

Random shuffling

```
for (int i = a.length - 1; i > 0; i--) {
    int j = rng.nextInt(i + 1);
    int temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
```

Each step fixes one position at the right end.

8

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Fisher–Yates shuffle. Assume rng is a java.util.Random instance. At each step choose j uniformly from 0 through i. Each permutation is equally likely under ideal independent uniform choices. Avoid choosing j from the full array on every iteration.

Circular shift left

```
if (a.length > 0) {
    int first = a[0];
    for (int i = 1; i < a.length; i++)
        a[i - 1] = a[i];
    a[a.length - 1] = first;
}
```

Before	10	20	30	40
After	20	30	40	10

9

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The source image shows wraparound, so this is a rotation. The guard handles an empty array. Copy left to right so unread source values remain available. A shift that discards data would need a specified fill value.

Circular shift right

```
if (a.length > 0) {
    int last = a[a.length - 1];
    for (int i = a.length - 1; i > 0; i--)
        a[i] = a[i - 1];
    a[0] = last;
}
```

Before	10	20	30	40
After	40	10	20	30

10

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Save the last element, then copy from right to left to avoid overwriting unread values. This example preserves every value through wraparound.

Enhanced for loop

```
for (double value : values) {
    System.out.println(value);
}

// Replace each array element using its index.
for (int i = 0; i < values.length; i++) {
    values[i] *= 2;
}
```

The loop variable receives an element value.

11

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

For primitive elements, assigning to the loop variable does not update an array element. For reference elements, a loop variable can refer to a mutable object and allow changes to that object. Use an index to replace elements or control traversal order.

Assignment shares the array

```
int[] a = {2, 3, 1};
int[] b = a;
b[0] = 99;
```

Expression	Result
a == b	true
a[0]	99
b[0]	99

12

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Both variables refer to the same int[] object after b = a. Changing an element through either reference changes the shared array. Reference: Oracle Java Tutorials, Arrays. https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

Copying elements into a new array

```
int[] source = {2, 3, 1, 5, 10};
int[] target = new int[source.length];
for (int i = 0; i < source.length; i++) {
    target[i] = source[i];
}

// Equivalent full-length copy:
int[] copy = Arrays.copyOf(source, source.length);
```

Import java.util.Arrays for the utility methods.

13

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Corrects the original sourceArrays.length typo. For primitive arrays, the copied values can be changed independently. A copy of a reference-type array still shares referenced objects. Reference: Java SE Arrays API. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html

Copying a range with arraycopy

```
int[] source = {2, 3, 1, 5, 10};
int[] target = new int[5];

System.arraycopy(source, 1, target, 0, 3);
```

Index	0	1	2	3	4
target	3	1	5	0	0

14

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

System.arraycopy(source, sourcePosition, destination, destinationPosition, count). The destination must already exist and both ranges must be in bounds. It supports overlapping ranges within the same array.

Passing an array to a method

```
static void printArray(int[] values) {
    for (int value : values)
        System.out.print(value + " ");
    System.out.println();
}

int[] numbers = {3, 1, 4};
printArray(numbers);
printArray(new int[] {8, 6, 7});
```

A parameter can accept any matching int[] array.

15

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The parameter receives a copy of the reference value. A method can inspect elements or update the shared array. The second call creates an anonymous array for the argument.

Java passes arguments by value

Inside a method	Effect visible to caller
Change an int parameter	Caller's int variable keeps its value
Change a[0]	Shared array element changes
Assign a = new int[3]	Caller's reference keeps its value

Changing an element and reassigning a reference differ.

16

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Reference: Oracle Java Tutorials, Passing Information to a Method or a Constructor.
https://docs.oracle.com/javase/tutorial/java/javaOO/arguments.html
For a primitive argument, the copied value is the primitive. For an array argument, the copied value is a reference. Reassigning the parameter changes only that local parameter.

Checkpoint: mutation and reassignment

```
static void change(int x, int[] b) {
    x = 9;
    b[0] = 99;
    b = new int[] {7, 8};
}

int n = 5;
int[] a = {1, 2};
change(n, a);
```

What are n and Arrays.toString(a) afterward?

17

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Answer: n remains 5, and a becomes [99, 2]. x is a copy of n. b initially refers to the same array as a, so b[0]=99 updates that array. Reassigning b later leaves a unchanged.

Linear search

```
static int linearSearch(int[] a, int key) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == key) return i;
    }
    return -1;
}
```

An unsorted array supports a direct sequential scan.

18

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The method returns the first matching index, or −1 when absent. It works with unsorted arrays. The body may inspect every element in the worst case.

Binary search: the sorted-array condition

Comparison	Next step
key < a[mid]	Continue below mid
key == a[mid]	Return mid
key > a[mid]	Continue above mid

The array must already be sorted.

19

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume ascending int[] input. Compare with the middle value, then discard the half that cannot contain the key. A duplicate key may return any matching index in the basic implementation.

Binary search: tracing key 60

```
[2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79]
```

Step	low	high	mid	a[mid]	Result
1	0	12	6	50	low = 7
2	7	12	9	66	high = 8
3	7	8	7	59	low = 8
4	8	8	8	60	Found

Each update removes mid and one side.

20

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Input from the original deck: [2,4,7,10,11,45,50,59,60,66,69,70,79]. Inclusive low/high bounds. First mid=6 (50), second mid=9 (66), third mid=7 (59), fourth mid=8 (60).

Binary search implementation

```
static int binarySearch(int[] a, int key) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

The loop ends when the candidate interval is empty.

21

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume sorted ascending input. The interval [low,high] contains every remaining candidate. Using low + (high − low)/2 avoids the potential overflow of (low + high)/2. Returning −1 is this teaching implementation’s contract; Arrays.binarySearch uses a different negative encoding.

Checkpoint: selecting a search

1 Which search works directly on an unsorted array?

2 With [2, 4, 6, 8, 10, 12, 14], where does the next interval lie when key = 6?

3 Why avoid sorting again before every query?

4 What should an unsuccessful search return?

22

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Answers: 1) Linear search unless sorting/preprocessing is justified. 2) mid=3, a[mid]=8, so high becomes 2. 3) Sorting on every query repeats unnecessary work. Sort once when appropriate, then search. 4) The custom methods return −1; Arrays.binarySearch returns a negative insertion-point encoding.

Selection sort: one pass at a time

State	Array
Start	29, 10, 14, 37, 13
After pass i = 0	10, 29, 14, 37, 13
After pass i = 1	10, 13, 14, 37, 29
After pass i = 2	10, 13, 14, 37, 29
After pass i = 3	10, 13, 14, 29, 37

The sorted prefix grows by one position per pass.

23

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Array values are an illustrative trace. At each step, choose the smallest element in the remaining suffix and swap it into position i. The finished prefix is sorted and contains the smallest values.

Selection sort as a method

```
static void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < a.length; j++) {
            if (a[j] < a[min]) min = j;
        }
        int temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
}
```

The caller's array contains the sorted values.

24

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The method sorts a in place. Using a min index keeps the search and swap separate. A self-swap is harmless when min equals i. The method handles empty and one-element arrays.

Sorting with java.util.Arrays

```
import java.util.Arrays;

double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
Arrays.sort(numbers);

char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
Arrays.sort(chars);
```

Arrays.sort changes the existing array.

Arrays.parallelSort offers a parallel alternative.

25

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Reference: Java SE Arrays API.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html
Arrays.sort mutates the passed array. Character sorting uses numeric character order. In this example, output is [4, A, D, F, P, a]. Arrays.parallelSort is available for relevant array types, but its benefit depends on input size, environment, and workload.

Displaying and comparing contents

```
int[] a = {2, 3, 1};
int[] b = {2, 3, 1};

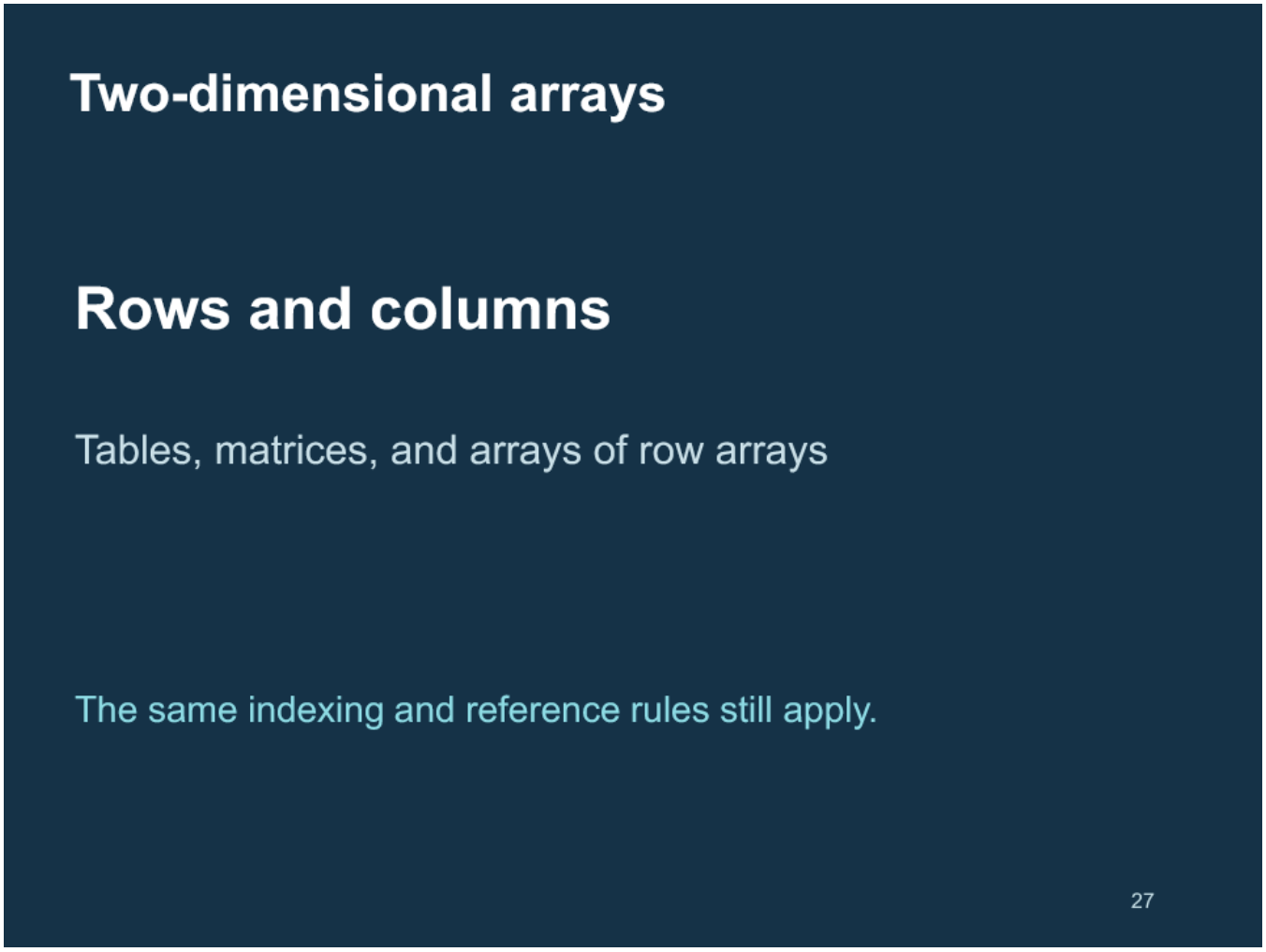
System.out.println(Arrays.toString(a)); // [2, 3, 1]
System.out.println(a == b);             // false
System.out.println(Arrays.equals(a,b)); // true
```

Printing the reference alone does not list its elements.

26

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Reference: Java SE Arrays API.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html
Arrays.toString formats a one-dimensional array. Arrays.equals compares corresponding elements. The == operator compares array reference identity. deepToString formats nested arrays; deepEquals compares nested contents.



Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Java represents a two-dimensional array as an array whose elements are row-array references. This section assumes non-null rows except when demonstrating incomplete allocation.

A distance table as array data

Miles	Chicago	Boston	New York	Atlanta	Miami	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
New York	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Miami	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0

distances[1][2] is 214 miles.

28

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

All distances in miles are reproduced from the supplied lecture as example data. City order: Chicago, Boston, New York, Atlanta, Miami, Dallas, Houston. These values are not a newly verified travel-distance dataset. Read row first, then column.

Declaring a two-dimensional array

```
int[][] matrix;              // declare
matrix = new int[3][4];       // create rows and values

int[][] grid = new int[3][4];
grid[0][0] = 3;
```

grid.length gives the number of rows.

grid[r].length gives the length of row r.

29

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

A table initializer

```
int[][] distances = {
    {   0,  983,  787,  714, 1375,  967, 1087},
    { 983,    0,  214, 1102, 1763, 1723, 1842},
    { 787,  214,    0,  888, 1549, 1548, 1627},
    { 714, 1102,  888,    0,  661,  781,  810},
    {1375, 1763, 1549,  661,    0, 1426, 1187},
    { 967, 1723, 1548,  781, 1426,    0,  239},
    {1087, 1842, 1627,  810, 1187,  239,    0}
};
```

The outer initializer contains the row initializers.

30

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The row and column order matches the distance table. Every row contains seven entries. Store distances in int values because the supplied example uses whole miles.

Initializing every element

```
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        matrix[r][c] = rng.nextInt(1000);
    }
}
```

The outer loop chooses a row.

The inner loop chooses an element within that row.

31

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume matrix and all rows are non-null, and rng is java.util.Random. The row-specific bound works for rectangular and ragged arrays. To read input, replace rng.nextInt(1000) with input.nextInt().

Row and column indexes

	Column 0	Column 1	Column 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12

matrix[2][1] selects row 2, then column 1.

32

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Example matrix values: {{1,2,3},{4,5,6},{7,8,9},{10,11,12}}. matrix[2][1] is 8. matrix.length is 4, matrix[0].length is 3.

Printing a two-dimensional array

```
for (int[] row : matrix) {
    for (int value : row)
        System.out.print(value + " ");
    System.out.println();
}

System.out.println(Arrays.deepToString(matrix));
```

The outer array's elements are row arrays.

33

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume matrix and all rows are non-null. The outer enhanced loop gives an int[] row reference. The inner loop reads values. Arrays.deepToString(matrix) can also format nested arrays.

Rectangular array lengths

```
int[][] matrix = new int[3][4];
```

Expression	Result
matrix.length	3
matrix[0].length	4
matrix[2].length	4
matrix[0][0]	0

34

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

new int[3][4] creates an outer array of length 3 and three int rows of length 4, initially filled with zero. matrix[0] is a row array, while matrix[0][0] is an int element.

Checkpoint: lengths and invalid indexes

```
int[][] a = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 11, 12}
};
```

a.length a[0].length a[3][2]

What happens when you evaluate a[4].length?

35

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Answers: a.length=4, a[0].length=3, a[3][2]=12. Accessing a[4].length throws ArrayIndexOutOfBoundsException because the outer array has indexes 0..3. A length is a count rather than a valid last index.

Ragged arrays

```
int[][] matrix = {
    {1, 2, 3, 4, 5},
    {2, 3, 4, 5},
    {3, 4, 5},
    {4, 5},
    {5}
};
```

Use matrix[r].length for each row's bound.

36

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Each row is an independent int[] with its own length. All rows here are non-null. The outer array has five rows; the row lengths are 5,4,3,2,1.

Allocating rows separately

```
int[][] triangle = new int[3][];

for (int r = 0; r < triangle.length; r++) {
    triangle[r] = new int[r + 1];
}
```

Row	Allocated length
triangle[0]	1
triangle[1]	2
triangle[2]	3

37

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

After new int[3][], the row references are null. Accessing a row length at that point causes NullPointerException. After the row allocations below, lengths are 1,2,3 and the int elements are zero.

Summing every element

```
long total = 0;
for (int[] row : matrix) {
    for (int value : row) {
        total += value;
    }
}
```

Every stored value contributes once.

38

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Works for non-null rectangular or ragged row arrays. For R rows and total M elements, the traversal costs Θ(R+M) including row-loop overhead. When every row is nonempty, this is Θ(M).

Summing each column

```
int columns = matrix[0].length;
for (int c = 0; c < columns; c++) {
    long sum = 0;
    for (int r = 0; r < matrix.length; r++) {
        sum += matrix[r][c];
    }
    System.out.println("column " + c + ": " + sum);
}
```

A column traversal keeps c fixed while r changes.

39

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume a non-null, nonempty rectangular matrix with C columns and non-null rows. For a ragged matrix, a column may be missing in some rows, so define a missing-column policy before using this code.

The row with the largest sum

```
int bestRow = -1;
long bestSum = Long.MIN_VALUE;
for (int r = 0; r < matrix.length; r++) {
    long sum = 0;
    for (int value : matrix[r]) sum += value;
    if (sum > bestSum) {
        bestSum = sum;
        bestRow = r;
    }
}
```

Compute one sum per row, then compare it.

40

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume at least one row and all rows non-null. Empty rows have sum 0 in this example. Using > keeps the earliest row when sums tie. Initializing the best sum to Long.MIN_VALUE handles all-negative row totals.

The first maximum in a matrix

```
int bestR = -1, bestC = -1;
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        if (bestR == -1 ||
                matrix[r][c] > matrix[bestR][bestC]) {
            bestR = r;
            bestC = c;
        }
    }
}
```

The result is a pair of indexes: row and column.

41

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume at least one stored element and all rows non-null. Row-major order visits increasing row, then increasing column. Using > keeps the earliest occurrence of a maximum. To find a minimum, use < and the corresponding initialization logic.

Shuffling a rectangular matrix

```
int rows = matrix.length, cols = matrix[0].length;
for (int i = rows * cols - 1; i > 0; i--) {
    int j = rng.nextInt(i + 1);
    int ir = i / cols, ic = i % cols;
    int jr = j / cols, jc = j % cols;
    int temp = matrix[ir][ic];
    matrix[ir][ic] = matrix[jr][jc];
    matrix[jr][jc] = temp;
}
```

Assume a nonempty rectangle with at least one column.

Convert a position into its row and column.

42

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Assume nonempty rectangular matrix, every row has the same positive length, rng is java.util.Random, and rows*cols fits an int. This applies Fisher–Yates to logical flattened positions without creating a flat copy. Row = position/cols and column = position%cols. The guard/policies for empty or ragged matrices would need separate handling.

Passing a two-dimensional array

```
static long sum(int[][] a) {
    long total = 0;
    for (int[] row : a)
        for (int value : row)
            total += value;
    return total;
}

long result = sum(matrix);
```

The parameter type is int[][].

43

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

The same pass-by-value rules apply. The parameter receives a copy of the outer-array reference. Changing a[r][c] changes the shared data. Reassigning only the parameter does not replace the caller’s variable. This method assumes all rows are non-null.

Review checkpoint

What makes an array index valid?

How does b = a differ from copying elements?

Why can a method mutate an array but fail to replace it?

What condition does binary search require?

Which length belongs in a ragged array’s inner loop?

44

Source: supplied Lecture01-Review.pptx, COMP 182, Lecture 1 review of COMP 110.

Answers: 1) Index valid in 0..length−1. 2) b=a shares an array, copying creates a new array of copied values. 3) Reassignment changes only the local parameter’s copied reference. 4) Sorted ascending input under the implementation shown. 5) matrix[r].length. Suggested practice: implement a method returning the smallest row sum, with a stated policy for an empty outer array.